QuakeFloat8 (QF8)

A Provably Optimal Log-Domain 8-Bit Format for ML Multiply-Accumulate

March 2026

Noah Everett

Harvard University

Abstract

QuakeFloat8 (QF8) is a block-scaled logarithmic 8-bit number format for machine learning in which multiplication reduces to 7-bit integer addition. The format uses a 1-bit sign, a 7-bit `u3.4` fixed-point $\log_2$ code per element, and a shared E8M0 block exponent per 32 elements.

Key results: (1) Log-uniform quantization is asymptotically minimax-optimal for NMSE over all source distributions (Theorem 3.5), with the approximation verified to 0.03% accuracy at $N = 256$. (2) The *equalization property* is validated empirically: across 1,481 weight tensors from 15 pretrained models spanning 5 architectures (GPT-2, OPT, Pythia, BERT, T5), QF8's SQNR is 38.1 ± 0.10 dB — effectively constant regardless of model, layer, or weight distribution (kurtosis correlation $r = -0.08$). (3) Weight-only post-training quantization from 124M to 7B parameters (GPT-2, Phi-2, Mistral-7B, Pythia-6.9B) shows QF8 degradation $\leq$0.03% on WikiText-2 perplexity, vs. +0.19–0.28% for MXFP8 E4M3 at 7B scale. W8A8 quantization reveals a known limitation: QF8's log-uniform spacing underperforms MXFP8 on activations that concentrate near zero. Quantization-aware training of GPT-2 Small (162M parameters) on OpenWebText shows QF8 tracking FP32 validation loss to within +0.016% through 40,000 steps ($\sim$11B tokens, val loss $\sim$3.1, perplexity $\sim$23). (4) The full QF8 MAC unit is estimated at $\sim$650 gates vs. $\sim$1350 for FP8 ($\sim$2$\times$ cheaper), with the multiply path itself 3.5$\times$ cheaper (66 vs. 231 gates). All gate counts are NAND2-equivalent estimates; no RTL synthesis has been performed.

Contents

1 Introduction and Motivation **3**
- 1.1 The 8-Bit Frontier . . . 3
- 1.2 The Quake Insight . . . 3
- 1.3 Relationship to Prior Work . . . 3
- 1.4 Contributions . . . 3

2 The QF8 Format **4**
- 2.1 Encoding Scheme . . . 4
- 2.2 Logarithmic Spacing . . . 4
- 2.3 Multiplication as Integer Addition . . . 5
- 2.4 Accumulation Strategy . . . 5
- 2.5 BMD Realizability . . . 5

3 Theoretical Results: Minimax Optimality **5**
- 3.1 Notation and Setup . . . 5
- 3.2 Relation to Classical Quantization Theory . . . 6
- 3.3 Theorem 2.1: Minimax NMSE Optimality . . . 7

4 Product Error Decomposition **8**
4.1 The General Formula . . . 8
4.2 Centroid Simplification . . . 9
4.3 Specialization to Log-Uniform Quantizers . . . 9
4.4 Optimality for Multiplication . . . 10
4.5 Extension to Dot Products . . . 10

5 Exact MSRE of Log-Uniform Quantization **10**

6 MSRE Separation and Format Comparison **11**
6.1 Log vs. Uniform: Exponential MSRE Separation . . . 11
6.2 QF8 vs. FP8 E4M3: Levels-per-Octave Advantage . . . 12

7 Hardware Cost Analysis **13**
7.1 Reference Data . . . 13
7.2 QF8 Multiply Path . . . 13
7.3 QF8-Medium MAC Unit . . . 13
7.4 Master Comparison . . . 14
7.5 Systolic Array Scaling (128×128) . . . 14
7.6 The Break-Even Analysis . . . 14
7.7 The Honest Bottom Line . . . 14

8 Empirical Evaluation **15**
8.1 SQNR on Pretrained Weights . . . 15
8.2 Cross-Model Equalization Experiment . . . 15
8.3 Post-Training Quantization on WikiText-2 . . . 16
8.3.1 Scaling to 7B Parameters . . . 17
8.4 Quantization-Aware Training . . . 19
8.5 Synthetic Benchmarks . . . 19

9 Literature Positioning **20**
9.1 Relationship to Johnson (2018) . . . 20
9.2 Relationship to MX Formats (OCP Microscaling) . . . 20
9.3 Relationship to NF4 (QLoRA) . . . 21
9.4 Logarithmic Number Systems . . . 21
9.5 What Is Genuinely Novel . . . 21
9.6 The One-Sentence Pitch . . . 22

10 Future Work **22**
10.1 Remaining Gaps . . . 22
10.2 Gradient Quantization . . . 23
10.3 BMD Decoder Conjecture . . . 23

11 Appendix: Rate-Distortion Context **23**
11.1 Gaussian Source Rate-Distortion . . . 23
11.2 Scalar Quantization Gap (Zador) . . . 23
11.3 Bit Allocation . . . 23

Introduction and Motivation

The 8-Bit Frontier

The push toward low-precision arithmetic for deep learning has been remarkably successful. FP16 training is standard practice; BFloat16 is ubiquitous; and 8-bit formats (INT8, FP8 E4M3/E5M2) are deployed in production hardware (NVIDIA H100, AMD MI300, Google TPU). The industry consensus, codified in the OCP Microscaling (MX) specification, is that 8 bits per element with block scaling is sufficient for both training and inference of large language models.

Yet a gap remains between the precision delivered by current 8-bit formats and what is theoretically achievable. FP8 E4M3 provides 8 representable levels per octave (factor of 2). Is this optimal? Can we do better at the same bit budget?

The Quake Insight

In 1999, the Quake III source code revealed a remarkable hack: the integer bit pattern of an IEEE 754 float is approximately a linear function of $\log_2(x)$. Reinterpreting float bits as integers, and vice versa, allows cheap approximate logarithmic arithmetic.

QF8 makes this accident intentional. By storing values as fixed-point $\log_2$ codes, multiplication becomes integer addition — replacing an $O(n^2)$-gate multiplier with an $O(n)$-gate adder. The format is paired with MX-style block scaling (E8M0 shared exponent per 32 elements) to achieve full FP32 dynamic range.

Relationship to Prior Work

QF8 builds on two prior lines of work. First, Johnson's (2018) ELMA format [1] demonstrated that log-domain multiply with Kulisch accumulation is viable for 8-bit deep learning arithmetic — QF8's core multiply-as-addition mechanism is Johnson's. Second, the OCP Microscaling (MX) specification [10] established E8M0 block scaling as a practical way to decouple dynamic range from per-element precision — QF8 adopts this directly. What QF8 adds is (a) a fixed-width `u3.4` encoding (vs. Johnson's variable-width posit regime), (b) formal minimax optimality analysis (the equalization property, Theorem 3.5), and (c) empirical validation at scale. A detailed comparison appears in Section 9.

Contributions

1. **Formal optimality.** We prove the *equalization property* of log-uniform quantization: MSRE $= \varepsilon^2/12$ for *any* source density, because the x^2 factors in the NMSE ratio cancel exactly. This density-independence implies that log-uniform quantization is the unique minimax-optimal quantizer for NMSE over all source distributions (Theorem 3.5), complementing Sun & Goyal's [16] variable-rate optimality result and extending it to the fixed-rate minimax setting. Log-uniform quantization also minimizes worst-case product error for multiply-accumulate (Theorem 4.4).

2. **Format design.** QF8 combines a `u3.4` fixed-point log code (16 levels per octave) with E8M0 block scaling, achieving 2× the precision per octave of FP8 E4M3 at the same effective storage cost (8.25 bits/element).

3. **Hardware analysis.** Gate-level estimates (NAND2-equivalent, not synthesized) show the QF8 multiply path at ∼66 gates vs. ∼231 for FP8 (3.5× cheaper). The complete QF8-Medium

MAC unit is estimated at ∼650 gates vs. ∼1350 for FP8 (∼2× cheaper), achieving near-INT8 area while providing floating-point dynamic range.

4. **Empirical validation.** The equalization property is confirmed across 1,481 weight tensors from 15 models and 5 architectures: QF8's SQNR is 38.1 ± 0.10 dB regardless of model or weight distribution. Weight-only PTQ from GPT-2 (124M) to Mistral-7B shows QF8 degradation ≤0.03% at 7B scale, vs. MXFP8's +0.19–0.28%. W8A8 quantization exposes a limitation: QF8's log-uniform spacing is suboptimal for activations concentrated near zero.

The QF8 Format

Encoding Scheme

Definition 2.1 (QF8 Encoding)**.** A QF8-encoded block of $k = 32$ values consists of:

- **Shared block scale** (8 bits): An E8M0 power-of-2 exponent $s \in \{2^{-127}, \ldots, 2^{127}\}$.
- **Per-element code** (8 bits each): 1 sign bit $\sigma \in \{0, 1\}$ and a 7-bit unsigned `u3.4` fixed-point code $c \in \{0, 1, \ldots, 127\}$.

The reconstruction formula is:

$$\hat{x} = (-1)^{\sigma} \cdot s \cdot 2^{(c-64)/16}, \qquad c \neq 0$$

with $c = 0$ encoding exact zero. The effective storage cost is $8 + 8/32 = 8.25$ bits per element.

Logarithmic Spacing

The `u3.4` code provides 3 integer bits and 4 fractional bits of $\log_2 |x/s|$, spanning 8 octaves with **16 representable levels per octave**. This is the *log-uniform quantizer*: the ratio between consecutive representable values is constant at $r = 2^{1/16} \approx 1.0443$, giving a maximum relative quantization error of

$$\frac{r-1}{2} \approx 2^{1/16} - 1 \approx 4.43\%.$$

Compare FP8 E4M3, which has 8 levels per octave (3-bit mantissa) and a maximum relative error of $1/16 = 6.25\%$ within each binade.

Remark (Why `u3.4`: Bit Allocation). The 7-bit log code admits several integer/fractional splits. The choice of 3 integer + 4 fractional bits is determined by the ML workload:

- **Per-element range:** 3 integer bits give $2^3 = 8$ octaves of per-element dynamic range. With E8M0 block scaling absorbing inter-block variation, the per-element code only needs to cover *within-block* spread. For typical ML tensors with block size 32, empirical within-block spread is 3–5 octaves; 8 octaves provides comfortable margin.
- **Precision:** 4 fractional bits give 16 levels per octave (MSRE $= 1.56 \times 10^{-4}$). Shifting one fractional bit to integer (`u4.3`: 16 octaves, 8 levels/octave) halves precision for already-sufficient range — strictly worse for weights and activations. Shifting one integer bit to fractional (`u2.5`: 4 octaves, 32 levels/octave) risks overflow for blocks with > 4 octave spread.
- **Gradient variant:** For backward passes, where gradients require wider dynamic range, a `u4.3` variant (16 octaves, 8 levels/octave) is the natural analog of FP8's E5M2 for gradients (Section 10).

Multiplication as Integer Addition

For two QF8 values with log codes c_a, c_b:

$$\text{sign}_{\text{product}} = \sigma_a \oplus \sigma_b \tag{1}$$

$$\text{log-code}_{\text{product}} = c_a + c_b \tag{2}$$

The product's log code is a simple 7-bit integer addition. In hardware, this replaces the 4×4 mantissa multiplier needed for FP8 E4M3 with a 7-bit carry-lookahead adder — an estimated $\sim$3.5$\times$ reduction in multiply-path gate count (66 vs. 231 gates). For the full MAC unit including accumulation, the estimated advantage is $\sim$2$\times$ (650 vs. 1350 gates; see Section 7).

Accumulation Strategy

The classical weakness of log-domain arithmetic is that *addition* in the original domain is expensive. QF8 adopts the **Kulisch accumulation** strategy (Johnson, 2018):

1. Multiply in log domain (integer addition of codes).
2. Convert each product to linear domain via a tiny $2^4 = 16$-entry lookup table.
3. Accumulate in a wide fixed-point register (32–48 bits for practical designs).

This hybrid log-multiply / linear-accumulate approach gets cheap multiplies *and* exact (or near-exact) accumulation.

BMD Realizability

Definition 2.2 (Bit-Manipulation Decodable (BMD))**.** A decoder $\hat{D} : \{0,1\}^B \to \mathbb{R}$ is *bit-manipulation decodable* if it can be expressed as a fixed-length straight-line program over $\{+, -, \times, \gg, \ll, \&, |, \oplus\}$ on machine-word integers, with the output reinterpreted as a float via type punning.

Theorem 2.3 (BMD Realizability)**.** *The log-uniform decode function $\hat{D}(n) = a \cdot r^n$ can be implemented via:*

1. ***Lookup table:*** *$O(N)$ storage, $O(1)$ read.*
2. ***Reinterpret cast (Quake trick):*** *$I = \alpha n + \beta$, then float reinterpret. Cost: 1 multiply + 1 add + type pun = 3 ops.*
3. ***Quadratic approximation:*** *$2^{e+f} = 2^e \cdot (1 + 0.6602f + 0.3398f^2)$. Cost: 2 multiplies + 2 adds + 1 shift = 5 ops. Maximum relative error: 0.27% (minimax optimal for constrained quadratic).*

Theoretical Results: Minimax Optimality

Notation and Setup

Let X be a positive random variable with pdf f supported on $[a, b] \subset \mathbb{R}_{>0}$. A B-bit scalar quantizer is a map $Q : [a, b] \to \hat{\mathcal{X}}$ with $N = 2^B$ reconstruction points partitioning $[a, b]$ into cells $\{S_k\}_{k=0}^{N-1}$ with codepoints $\{c_k\}$.

Definition 3.1 (Distortion Measures)**.**

$$\delta_X = X - Q(X), \qquad \varepsilon_X = \delta_X / X \tag{3}$$

$$\mathrm{MSE}_X = \mathbb{E}[\delta_X^2], \qquad \mathrm{NMSE}_X = \frac{\mathbb{E}[\delta_X^2]}{\mathbb{E}[X^2]} \tag{4}$$

$$\mathrm{MSRE}_X = \mathbb{E}[\varepsilon_X^2], \qquad \mathrm{SQNR} = 10\log_{10}(1/\mathrm{NMSE})\ \mathrm{dB} \tag{5}$$

Definition 3.2 (Auxiliary Quantities)**.**

$$\alpha_X = \mathbb{E}[X\delta_X]/\mathbb{E}[X^2] \qquad \text{(signal-error correlation)} \tag{6}$$

$$\beta_X = \mathbb{E}[Q(X)\delta_X]/\mathbb{E}[X^2] \qquad \text{(reconstruction-error correlation)} \tag{7}$$

$$\gamma_X = \mathbb{E}[Q(X)^2]/\mathbb{E}[X^2] \qquad \text{(quantized power ratio)} \tag{8}$$

$$\rho_X = \mathbb{E}[X \cdot Q(X)]/\mathbb{E}[X^2] \qquad \text{(signal-quantized correlation)} \tag{9}$$

Algebraic identities (no assumptions required):

$$\alpha_X = \beta_X + \mathrm{NMSE}_X, \qquad \gamma_X = 1 - 2\alpha_X + \mathrm{NMSE}_X, \qquad \rho_X = 1 - \alpha_X \tag{10}$$

Definition 3.3 (Centroid Condition)**.** A quantizer satisfies the centroid condition if $\mathbb{E}[\delta_X \mid X \in S_k] = 0$ for every cell. This holds for Lloyd-Max quantizers and approximately for any well-designed quantizer in the high-resolution regime. Under the centroid condition: $\beta_X = 0$, $\alpha_X = \mathrm{NMSE}_X$, and $\gamma_X = 1 - \mathrm{NMSE}_X$.

Definition 3.4 (Log-Uniform Quantizer)**.** The log-uniform quantizer with N levels over dynamic range $[a, b]$ has cell boundaries $a \cdot r^k$ for $k = 0, \ldots, N$, where $r = (b/a)^{1/N} = 2^{R/N}$ and $R = \log_2(b/a)$ is the dynamic range in octaves. With geometric midpoint codepoints $c_k = a \cdot r^{k+1/2}$, the relative cell width is constant:

$$\frac{w(x)}{x} = r - 1 = e^{\varepsilon} - 1 \approx \varepsilon, \qquad \varepsilon = \frac{R \ln 2}{N} \tag{11}$$

Relation to Classical Quantization Theory

The classical companding literature studies optimal nonuniform quantizers under MSE distortion. Bennett [13] established the foundational high-resolution framework: for a compander with local step size $w(x)$ and source pdf f, the distortion is $D = \frac{1}{12}\int w(x)^2 f(x)\,dx$, giving the $\varepsilon^2/12$ approximation for uniform quantizers. Panter & Dite [14] derived the MSE-optimal compander (the $p(x)^{1/3}$ one-third-power rule): it achieves equal MSE contribution per cell, but depends critically on the source distribution. The standard textbook treatments [17, 18] cover these results thoroughly.

For *relative* error metrics, Smith [15] observed that logarithmic companding achieves approximately amplitude-independent SNR for PCM telephony — meaning the SNR is roughly constant as a function of *signal level* (e.g., sinusoidal amplitude). This is structurally distinct from the equalization property of Theorem 3.5: Smith's result is about signal amplitude, not about source probability density. Indeed, Smith's distortion formula retains a distribution-dependent factor (e.g., 0.707 for exponential, 0.798 for Gaussian distributions), so the distortion *is* distribution-dependent in his framework.

Most directly related, Sun & Goyal [16] study scalar quantization explicitly under the MSRE metric. They prove that logarithmic companding is *variable-rate*-optimal for MSRE: given a source distribution f, the entropy-constrained optimal quantizer is logarithmic. However, they also show

that logarithmic companding is *not* generally optimal for fixed-rate quantization — the optimal fixed-rate MSRE quantizer depends on the source distribution.

Theorem 3.5 provides a complementary and stronger fixed-rate result: the log-uniform quantizer achieves MSRE $= \varepsilon^2/12$ for *any* source density (the equalization property), and this constant value is the fixed-rate minimax optimum. The key structural insight is the x^2 cancellation in the NMSE ratio, which is absent in all prior companding results: it makes the log-uniform quantizer's error density-independent rather than merely amplitude-independent. The minimax framing is not addressed in any prior work found.

Why minimax for ML? The ML setting motivates worst-case (minimax) rather than average-case (Bayes) analysis for two reasons. First, weight and activation distributions shift substantially across layers, training stages, and models, so a quantizer optimal for one distribution may be poor for another. Second, the equalization property has direct practical value: for *weight* quantization, a format with density-independent NMSE requires no per-layer calibration of the quantizer grid, and its error bound holds regardless of whether the layer distribution matches any assumed prior. (For activations, whose distributions may concentrate near zero, the log-uniform grid's precision allocation is suboptimal; see Section 8.) The log-uniform quantizer's minimax value $\varepsilon^2/12$ is not just a worst-case bound — by the equalization property, it is achieved for every source distribution, including Gaussian, uniform, Laplace, and the empirical distributions observed in trained networks.

Theorem 2.1: Minimax NMSE Optimality

Theorem 3.5 (Minimax NMSE Optimality of Log-Uniform Quantization)**.** *In the high-resolution regime ($N \gg 1$), among all N-level quantizers on $[a, b]$, the log-uniform quantizer uniquely minimizes the worst-case NMSE (and MSRE) over all densities:*

$$Q^*_{\log} = \arg\min_{Q \in \mathcal{Q}_N} \max_{f \in \mathcal{F}[a,b]} \mathrm{NMSE}(Q, f)$$

The minimax value is:

$$\boxed{\mathrm{NMSE}^* = \frac{\varepsilon^2}{12} + O(\varepsilon^4) = \frac{(R \ln 2)^2}{12N^2}} \tag{12}$$

For $B = 8$ bits, $R = 16$ octaves: $\mathrm{NMSE}^* = 1.564 \times 10^{-4}$ *(*$\mathrm{SQNR} = 38.1$ *dB).*

Proof. **Step 1 (Density-independence).** For the log-uniform quantizer, $w(x) = x \cdot \varepsilon + O(x\varepsilon^2)$. Under the high-resolution approximation:

$$\mathrm{NMSE} = \frac{\int_a^b \frac{w(x)^2}{12} f(x)\, dx}{\int_a^b x^2 f(x)\, dx} = \frac{\frac{\varepsilon^2}{12} \int_a^b x^2 f(x)\, dx}{\int_a^b x^2 f(x)\, dx} = \frac{\varepsilon^2}{12}$$

The x^2 factors cancel exactly. Similarly, MSRE $= \varepsilon^2/12$ for any density f. This is the *equalization property* of log-uniform quantization.

This is confirmed by exhaustive computation over three test densities (log-uniform, uniform, $f \propto x^2$) with $N = 256$, $R = 16$: all give $\mathrm{NMSE} = 1.564 \times 10^{-4}$ within $< 0.1\%$ deviation.

Step 2 (Non-constant ϕ is worse for some density). Define $\phi(x) = w(x)/x$ (relative cell width). For any non-log-uniform quantizer, ϕ is not constant. Let $x^* = \arg\max_x \phi(x)$ and $\phi^* = \phi(x^*)$.

We first show $\phi^* > \varepsilon$. Any N-cell quantizer on $[a, b]$ satisfies the coverage constraint $\int_a^b dx/w(x) = N$, i.e.

$$\int_a^b \frac{dx}{\phi(x)\, x} = N. \tag{13}$$

For the log-uniform quantizer $\phi \equiv \varepsilon$, this gives $(1/\varepsilon)\ln(b/a) = N$ since $\varepsilon = R\ln 2/N$ and $\ln(b/a) = R\ln 2$. Suppose for contradiction that ϕ is non-constant but $\phi(x) \le \varepsilon$ everywhere. Then $1/\phi(x) \ge 1/\varepsilon$ everywhere, with strict inequality on a set of positive measure, so

$$N = \int_a^b \frac{dx}{\phi(x)\,x} > \int_a^b \frac{dx}{\varepsilon\,x} = \frac{\ln(b/a)}{\varepsilon} = N,$$

a contradiction. Therefore $\phi^* > \varepsilon$ for any non-log-uniform quantizer.

Since ϕ is piecewise constant on finitely many cells, its maximum is achieved at some interior point x^* of some cell S_{k^*} (cell boundaries are shared between two cells, each with a well-defined constant ϕ value; the maximum over finitely many values is achieved in at least one cell's interior). For any $\eta > 0$ small enough that $[x^* - \eta, x^* + \eta] \subset S_{k^*}$, the uniform density on $[x^* - \eta, x^* + \eta]$ achieves:

$$\text{MSRE} \ge \frac{\phi(x^*)^2}{12} - g(\eta), \qquad g(\eta) \to 0 \text{ as } \eta \to 0$$

(Here $g(\eta) \to 0$ because the density is supported within the single cell S_{k^*} where $\phi \equiv \phi^*$, and the high-resolution MSRE over that cell is exactly $\phi^{*2}/12$.) Therefore $\max_f \text{MSRE}(Q, f) \ge (\phi^*)^2/12 > \varepsilon^2/12 = \text{MSRE}^*$.

Step 3 (Uniqueness). If $Q \ne Q_{\log}$, then ϕ is non-constant and Step 2 gives strict inequality. □ □

Corollary 3.6 (Minimax MSRE)**.** *The same quantizer also minimizes worst-case MSRE. The proof is identical, with $w(x)^2 f(x)/x^2$ replacing $w(x)^2 f(x)/\mathbb{E}[X^2]$.*

Remark (High-Resolution Regime)*.* The proof uses Bennett's high-resolution integral approximation, which is asymptotically exact as $N \to \infty$. Both the equalization property (Step 1) and the coverage constraint argument (Step 2) operate in this continuous limit. The theorem is therefore an *asymptotic* result. For finite N, Theorem 5.1 verifies that the exact per-cell MSRE matches $\varepsilon^2/12$ to within 0.03% at $N = 256$ (Table 1), confirming that the approximation is excellent for practical quantizer sizes. However, we have not formally proved that the minimax *ordering* (log-uniform beats all alternatives) holds for small N; the finite-N evidence is numerical, not analytic.

Remark (Domain Restriction and Zero Handling)*.* Theorem 3.5 applies to densities on $[a, b] \subset \mathbb{R}_{>0}$, excluding zero. In practice, ML tensors contain exact zeros (e.g., ReLU outputs) and near-zero values. QF8 handles this in two ways: (1) the code $c = 0$ represents exact zero, absorbing any point mass at the origin; (2) block scaling with E8M0 exponents as small as 2^{-127} shifts the smallest nonzero representable value to $\sim 5.9 \times 10^{-39}$, far below any practical threshold. The minimax guarantee applies to the nonzero portion of the signal; the zero code handles the rest exactly.

Product Error Decomposition

This section addresses how quantization error propagates through multiplication — the core operation in neural network inference.

The General Formula

Theorem 4.1 (General Product Error Decomposition)**.** *Let X, Y be independent positive random variables with quantizers Q_X, Q_Y. Then:*

$$\boxed{\text{NMSE}_{\text{prod}} = \text{NMSE}_X + \text{NMSE}_Y + \text{NMSE}_X \cdot \text{NMSE}_Y + 2\alpha_X\alpha_Y - 2\alpha_X \cdot \text{NMSE}_Y - 2\text{NMSE}_X \cdot \alpha_Y} \tag{14}$$

where $\alpha_X = \mathbb{E}[X\delta_X]/\mathbb{E}[X^2]$. *Equivalently:*

$$\mathrm{NMSE}_{\mathrm{prod}} = 1 - 2\rho_X\rho_Y + \gamma_X\gamma_Y \tag{15}$$

This is exact, using only $X \perp Y$. *No high-resolution, centroid, or distributional assumptions.*

Proof. **Step 1.** Decompose the product error:

$$XY - Q_XQ_Y = (Q_X + \delta_X)(Q_Y + \delta_Y) - Q_XQ_Y = \underbrace{Q_X\delta_Y}_{A} + \underbrace{\delta_XQ_Y}_{B} + \underbrace{\delta_X\delta_Y}_{C}$$

Step 2. Expand $\mathbb{E}[(A+B+C)^2]$ using $X \perp Y$. Since Q_X, δ_X depend only on X and Q_Y, δ_Y depend only on Y, every product of an X-function and a Y-function factors:

$$\begin{aligned}\mathbb{E}[(A+B+C)^2] = {} & \mathbb{E}[Q_X^2]\mathbb{E}[\delta_Y^2] + \mathbb{E}[Q_Y^2]\mathbb{E}[\delta_X^2] + \mathbb{E}[\delta_X^2]\mathbb{E}[\delta_Y^2] \\ & + 2\mathbb{E}[Q_X\delta_X]\mathbb{E}[Q_Y\delta_Y] + 2\mathbb{E}[Q_X\delta_X]\mathbb{E}[\delta_Y^2] + 2\mathbb{E}[\delta_X^2]\mathbb{E}[Q_Y\delta_Y]\end{aligned}$$

Step 3. Divide by $\mathbb{E}[X^2]\mathbb{E}[Y^2]$ and substitute $\beta_X = \alpha_X - \mathrm{NMSE}_X$, $\gamma_X = 1 - 2\alpha_X + \mathrm{NMSE}_X$:

$$\mathrm{NMSE}_{\mathrm{prod}} = \gamma_X\mathrm{NMSE}_Y + \gamma_Y\mathrm{NMSE}_X + \mathrm{NMSE}_X\mathrm{NMSE}_Y + 2\beta_X\beta_Y + 2\beta_X\mathrm{NMSE}_Y + 2\mathrm{NMSE}_X\beta_Y$$

Expanding $\beta = \alpha - \mathrm{NMSE}$ and simplifying yields (14). □ □

Centroid Simplification

Corollary 4.2 (Product Error Under Centroid Condition)**.** *If both quantizers satisfy the centroid condition* ($\alpha_X = \mathrm{NMSE}_X$, $\alpha_Y = \mathrm{NMSE}_Y$)*, then:*

$$\boxed{1 - \mathrm{NMSE}_{\mathrm{prod}} = (1 - \mathrm{NMSE}_X)(1 - \mathrm{NMSE}_Y)} \tag{16}$$

Equivalently: $\mathrm{NMSE}_{\mathrm{prod}} = \mathrm{NMSE}_X + \mathrm{NMSE}_Y - \mathrm{NMSE}_X \cdot \mathrm{NMSE}_Y$.

Proof. Substitute $\alpha_X = n_X$, $\alpha_Y = n_Y$ into (14): $n_X + n_Y + n_Xn_Y + 2n_Xn_Y - 2n_Xn_Y - 2n_Xn_Y = n_X + n_Y - n_Xn_Y$. □ □

Remark (Interpretation of the Cross-Term). The sign of the cross-term is **minus**: $\mathrm{NMSE}_{\mathrm{prod}} = n_X + n_Y - n_Xn_Y$. The identity $(1 - n_X)(1 - n_Y) = 1 - n_{\mathrm{prod}}$ has a clean interpretation: the signal preservation fraction of the product equals the product of individual preservation fractions. Since $n_Xn_Y = O(1/N^4)$ while $n_X + n_Y = O(1/N^2)$, the cross-term is negligible in practice. Both forms (14) and (15) have been confirmed by exhaustive computation over 14,641 grid points covering all combinations of $(\mathrm{NMSE}_X, \mathrm{NMSE}_Y) \in \{0, 0.01, \ldots, 1\}^2$.

Specialization to Log-Uniform Quantizers

Theorem 4.3 (Log-Uniform Product Error)**.** *For log-uniform quantizers with geometric midpoints,* $\mathrm{MSRE} = \mathrm{NMSE} = \varepsilon^2/12$ *for any density, and:*

$$\mathrm{NMSE}_{\mathrm{prod}} = \frac{(R\ln 2)^2}{6N^2} + O(1/N^4) = 2\mathrm{NMSE}^* + O(1/N^4) \tag{17}$$

The original MSRE-based formula gives the correct answer to leading order for log-uniform quantizers (despite being invalid in general) because the equalization property makes $\mathrm{MSRE} \approx \mathrm{NMSE}$ *with ratio* 1.0009 *for* $N = 16$.

Optimality for Multiplication

Theorem 4.4 (Log-Uniform Optimal for Multiplication)**.** *Among all N-level centroid-satisfying quantizer pairs (Q_X, Q_Y) on $[a, b]$, the log-uniform quantizer minimizes the worst-case product NMSE:*

$$Q_X^* = Q_Y^* = Q_{\log} = \arg\min_{Q_X, Q_Y \in \mathcal{Q}_N^c} \max_{f_X, f_Y} \mathrm{NMSE}_{\mathrm{prod}}$$

where $\mathcal{Q}_N^c$ denotes centroid-satisfying N-level quantizers. The minimax value is $2n^* - (n^*)^2 = 1 - (1 - n^*)^2$ where $n^* = \varepsilon^2/12$.

Proof. Uses two properties: (1) *Monotonicity*: under the centroid condition, $\mathrm{NMSE}_{\mathrm{prod}} = n_X + n_Y - n_X n_Y$, and $\partial/\partial n_X(n_X + n_Y - n_X n_Y) = 1 - n_Y > 0$. (2) *Density-independence*: $Q_{\log}$ achieves NMSE $= n^*$ for all densities. For any $Q_X \neq Q_{\log}$, Theorem 3.5 provides f_X^* with $\mathrm{NMSE}(Q_X, f_X^*) > n^*$, giving strictly worse product NMSE. □

Remark (Centroid Restriction). The centroid condition ($\mathbb{E}[\delta_X \mid X \in S_k] = 0$) is without loss of generality for this optimization: given any quantizer Q that violates it, replacing each codepoint by its conditional mean yields a quantizer Q' with strictly lower NMSE and the same cell partition. The minimax over centroid quantizers therefore equals the minimax over all quantizers.

Extension to Dot Products

Corollary 4.5 (Dimension-Free Dot Product NMSE)**.** *For $Z = \sum_{i=1}^d w_i x_i$ with independent pairs, iid within type, and centroid-condition quantizers:*

$$\mathrm{NMSE}_Z \approx \mathrm{NMSE}_w + \mathrm{NMSE}_x + O(1/N^4) \tag{18}$$

The NMSE does not grow with dimension d.

Proof sketch. Under independence across dimensions: $\mathbb{E}[(Z - \hat{Z})^2] = d \cdot \mathbb{E}[w^2 x^2](n_w + n_x - n_w n_x)$ and $\mathbb{E}[Z^2] = d \cdot \mathbb{E}[w^2 x^2]$. Division yields the result. □

Remark (Independence Assumption). The dimension-free result requires independence of the pairs (w_i, x_i) across dimensions i. This holds for standard weight–activation products in feedforward layers, where weights are fixed and activations depend on independent input features. However, it does *not* hold in attention mechanisms, where queries q_i and keys k_j are derived from the same input sequence and may be correlated. In practice, the NMSE degradation from correlated pairs is bounded by the product error formula (14) applied per-element, so the dimension-free approximation remains a useful guide even when independence is approximate.

Exact MSRE of Log-Uniform Quantization

Theorem 5.1 (Exact Per-Cell MSRE)**.** *For a log-uniform quantizer cell with common ratio $r = 2^{R/N}$ and geometric midpoint codepoint, the exact MSRE (under uniform intra-cell density) is:*

$$\mathrm{MSRE}_k = 2 - \frac{2\sqrt{r}\,\ln r}{r - 1} \tag{19}$$

For small $\varepsilon = \ln r = R \ln 2/N$:

$$\boxed{\mathrm{MSRE}_k = \frac{\varepsilon^2}{12} - \frac{7\varepsilon^4}{2880} + O(\varepsilon^6)} \tag{20}$$

Proof. For a cell $[a, ra)$ with geometric midpoint $c = a\sqrt{r}$, substituting $u = x/a$:

$$\mathrm{MSRE}_k = \frac{1}{r-1}\int_1^r \left(1 - \frac{\sqrt{r}}{u}\right)^2 du = \frac{2(r-1) - 2\sqrt{r}\ln r}{r-1} = 2 - \frac{2\sqrt{r}\,\ln r}{r-1}$$

For the Taylor expansion, substitute $r = e^\varepsilon$:

$$\frac{2\sqrt{r}\,\ln r}{r-1} = \frac{2\varepsilon\, e^{\varepsilon/2}}{e^\varepsilon - 1} = \frac{2(1 + \varepsilon/2 + \varepsilon^2/8 + \cdots)}{1 + \varepsilon/2 + \varepsilon^2/6 + \cdots} = 2\left(1 - \frac{\varepsilon^2}{24} + O(\varepsilon^3)\right)$$

Therefore $\mathrm{MSRE}_k = 2 - (2 - \varepsilon^2/12) = \varepsilon^2/12 + O(\varepsilon^4)$. This *confirms* the high-resolution approximation. □ □

Remark (Midpoint Choice: Geometric vs. Harmonic). The format definition and Theorem 5.1 use geometric midpoint codepoints $c_k = ar^{k+1/2} = \sqrt{q_k q_{k+1}}$, which minimize per-cell MSRE under uniform intra-cell density. An alternative is the harmonic mean $c_k = 2q_k q_{k+1}/(q_k + q_{k+1})$, which minimizes the worst-case *absolute* relative error within each cell to $(r-1)/(r+1)$ — a quantity that is exactly constant across bins without any high-resolution approximation. The accompanying Lean 4 numerical verification uses harmonic midpoints for this reason. The two midpoints agree to first order: $c_k^{\mathrm{geo}} = c_k^{\mathrm{harm}}(1 + O(\varepsilon^2))$, and both yield $\mathrm{MSRE} = \varepsilon^2/12 + O(\varepsilon^4)$. The difference appears only in the $O(\varepsilon^4)$ correction and is negligible at $N = 256$ ($< 0.2\%$ relative). Crucially, the minimax optimality of log-uniform *spacing* (Theorem 3.5) is independent of the midpoint rule: it depends only on cell widths $w(x)/x$, not on the choice of reconstruction point within each cell.

Table 1: Numerical verification: exact MSRE vs. approximations.

ε	Exact MSRE	$\varepsilon^2/12$	$\varepsilon^2/2$ (wrong)	Exact / ($\varepsilon^2/12$)
0.01	8.333×10^{-6}	8.333×10^{-6}	5.000×10^{-5}	1.000
0.05	2.083×10^{-4}	2.083×10^{-4}	1.250×10^{-3}	0.9999
0.10	8.331×10^{-4}	8.333×10^{-4}	5.000×10^{-3}	0.9997
0.20	3.329×10^{-3}	3.333×10^{-3}	2.000×10^{-2}	0.9989

MSRE Separation and Format Comparison

Log vs. Uniform: Exponential MSRE Separation

Theorem 6.1 (Exponential Separation Under MSRE)**.** *For $X \sim \mathrm{LogNormal}(0, \sigma^2)$ quantized to N levels:*

(i) ***Under MSRE:*** *the ratio of uniform quantization MSRE to logarithmic quantization MSRE is exponential in σ^2:*

$$\frac{\mathrm{MSRE}_{\mathrm{uniform}}}{\mathrm{MSRE}_{\mathrm{log}}} = e^{\Theta(\sigma^2)} \tag{21}$$

(ii) ***Under MSE:*** *the two quantizers achieve comparable distortion (ratio ≈ 1).*

This result is significant because MSRE is the natural metric for floating-point systems — it measures relative accuracy, which is what determines model quality.

Table 2: MSRE and MSE ratios (uniform / log) for LogNormal$(0, \sigma^2)$, $N = 256$.

σ	MSE ratio	MSE dB	MSRE ratio	MSRE dB
1.00	0.98	−0.1	8.5	9.3
1.50	0.99	−0.0	1,520	31.8
2.00	0.99	−0.0	346,000	55.4
2.77	1.00	0.0	3.0×10^9	94.8

Proof sketch. For $X = e^Z$ with $Z \sim \mathcal{N}(0, \sigma^2)$: Log quantization on X corresponds to uniform quantization on Z (Gaussian), giving $\text{MSRE}_{\text{log}} \approx \varepsilon^2/12$. Uniform quantization on X translates to highly nonuniform quantization on $Z = \ln X$: at $Z = k\sigma$, the local step in log-space grows as $\propto e^{2k\sigma}$, and integrating against the Gaussian density yields $\text{MSRE}_{\text{uniform}} \propto e^{c\sigma^2}$. MSE is insensitive because it is dominated by absolute error on large values where both quantizers are comparable. □

Remark (Scope of the Separation Theorem). Theorem 6.1 compares log-uniform quantization against *truly uniform* (fixed-step) quantization — the regime of INT8. FP8 E4M3 is *not* a uniform quantizer: within each binade it has 8 uniformly-spaced levels, but across binades the step size doubles, making it quasi-logarithmic at coarse scale. Consequently, the exponential MSRE separation of Theorem 6.1 applies to the QF8-vs-INT8 comparison under MSRE, not to the QF8-vs-FP8 comparison.

The +6.6 dB advantage of QF8 over FP8 E4M3 (Section 6.2) has a different, simpler origin: QF8 provides 16 levels per octave vs. FP8's 8, giving a step-size ratio of 2 and an MSRE ratio of 4 ($10 \log_{10}(4) = 6.0$ dB). The remaining ∼0.6 dB arises from FP8's piecewise-uniform spacing within each binade, which incurs a small additional penalty relative to purely log-uniform spacing (Table 3).

QF8 vs. FP8 E4M3: Levels-per-Octave Advantage

Table 3: Within-binade precision comparison.

Metric	IEEE E4M3	Log-uniform (8/oct)	Ratio	dB gap
Average MSRE (per binade)	6.510×10^{-4}	6.256×10^{-4}	1.041	0.2
Peak relative error	0.0625	0.04427	1.41	—
Worst-case MSRE (HR)	1.302×10^{-3}	6.256×10^{-4}	2.08	3.18

Key Result

QF8 achieves SQNR = 38.1 dB vs. FP8 E4M3's 31.5 dB on $\mathcal{N}(0, 1)$ — a **6.6 dB advantage**, corresponding to a ∼4.6× reduction in quantization noise power. This advantage is structural (16 vs. 8 levels per octave) and holds across all tested distributions.

Format	SQNR on $\mathcal{N}(0, 1)$	Gap from QF8
QF8 (log-uniform, 256 levels)	38.1 dB	—
FP8 E4M3 (IEEE)	31.5 dB	6.6 dB worse

Remark. The FP8 E4M3 SQNR of 31.5 dB is verified by per-binade analysis across all 126 positive representable values, correcting an earlier figure of 25.1 dB that arose from an error in the dynamic-range assumption.

Hardware Cost Analysis

This section presents gate-level hardware models anchored to published silicon data from Horowitz (ISSCC 2014), Johnson (2018 ELMA ASIC at 28nm), and NVIDIA architecture whitepapers. **All gate counts are NAND2-equivalent estimates.** No RTL has been written and no synthesis has been performed. The numbers should be treated as order-of-magnitude guidance, not as measured silicon results.

Reference Data

Horowitz ISSCC 2014 (45nm CMOS): 8-bit integer multiply = 0.2 pJ, 8-bit add = 0.03 pJ, 32-bit FP multiply = 3.7 pJ.

Johnson ELMA synthesis (28nm): 8-bit ELMA/38-bit Kulisch = 0.96× power, 1.12× area vs. INT8/32-bit MAC. 16-bit ELMA = 0.59× power, 0.68× area vs. FP16 FMA.

QF8 Multiply Path

The core QF8 advantage: multiplication reduces to a 7-bit integer addition plus sign XOR.

Table 4: QF8 multiply-path gate count.

Component	Gates	Area (μm^2, 7nm)
Sign XOR	1	0.04
7-bit CLA adder	50	2.0
Overflow/saturation detect	5	0.2
Zero detection	10	0.4
Total: multiply	**66**	**2.6**

Compare: INT8 8×8 multiplier = 400 gates; FP8 4×4 mantissa multiplier + exponent logic = 230 gates; FP16 11×11 multiplier = 850 gates; FP32 24×24 multiplier = 3200 gates. The QF8 multiply path is 3.5× **to** 48× **cheaper** than alternatives.

QF8-Medium MAC Unit

The recommended design point: 6-bit barrel shifter and 32-bit carry-save Kulisch accumulator for intra-block accumulation, with block-pair results reduced via FP32 arithmetic (matching MX-format accelerator practice).

Table 5: QF8-Medium MAC unit breakdown (7nm, 1 GHz).

Component	Gates	Area (μm^2)
Multiply path (Table 4)	66	2.6
exp2 LUT (16 × 12-bit ROM)	100	4.0
6-bit barrel shifter	150	6.0
32-bit carry-save Kulisch accumulator	300	12.0
Control/mux	34	1.4
Total	**650**	**26.0**

Table 6: Per-MAC unit comparison at 7nm, 1 GHz.

Metric	INT8	FP8 E4M3	FP16	FP32	**QF8-Med**
Gate count	750	1,350	2,400	5,950	**650**
Area (μm^2)	30	54	96	238	**26**
Energy (pJ/MAC)	0.040	0.070	0.130	0.350	**0.033**
Pipeline stages	2	3	3	3–4	**2**
Critical path (gates)	24–30	35–45	38–48	55–70	**20–26**

Master Comparison

Systolic Array Scaling (128 × 128)

The Break-Even Analysis

The log-domain advantage scales differently with bit-width because the multiplier is $O(n^2)$ while the adder is $O(n)$:

The crossover is at ∼7–8 bits. Below 7 bits, the LUT and accumulator overhead exceeds multiplier savings. Above 8 bits, log-domain wins decisively and the advantage grows quadratically. This confirms Johnson's 2018 finding that 16-bit ELMA achieves 41% power reduction over FP16.

The Honest Bottom Line

- **QF8 vs FP32/FP16:** Clear, large, robust win (3–10× savings).
- **QF8 vs FP8:** Clear win (∼2× area and power savings).
- **QF8 vs INT8:** Slight win (0.87× area, 0.83× power). The carry-save Kulisch design tips the balance in QF8's favour.
- **The real QF8 value is 2× precision-per-octave at INT8-equivalent hardware cost and half the cost of FP8.**

System-level efficiency. The per-MAC energy advantages above apply to the compute datapath only. Modern LLM inference is typically memory-bandwidth-bound, where data-movement energy dominates arithmetic cost by two orders of magnitude [6]. In this regime, QF8's primary efficiency contribution is indirect: at identical 8-bit bandwidth, QF8 delivers +6.6 dB higher SQNR than FP8 E4M3, potentially enabling 8-bit deployment in settings that would otherwise require 16-bit precision—halving memory traffic and the associated energy cost. Validating this claim requires

Table 7: QF8-Medium savings ratios vs. each baseline.

Metric	vs FP32	vs FP16	vs FP8	vs INT8
Area savings	**89%**	**73%**	**52%**	**13%**
Power savings	**91%**	**75%**	**53%**	**17%**
Latency improvement	55–65%	35–45%	25–35%	∼5–10%

Table 8: 128×128 systolic array estimates at 7nm, 1 GHz.

Metric	INT8	FP8	FP16	FP32	**QF8-Med**
Compute area (mm^2)	0.49	0.88	1.57	3.90	**0.43**
Total power (W)	0.68	1.19	2.21	5.93	**0.56**
TOPS/W	48.2	27.6	14.8	5.5	**58.5**
TOPS/mm^2	66.9	37.3	20.9	8.4	**76.2**

demonstrating QF8 at 8 bits matching FP16 quality on production-scale models, which we leave to future work.

Empirical Evaluation

All experiments use identical block-scaled quantization ($k = 32$ elements) with E8M0 shared exponents for all three formats (QF8, MXFP8 E4M3, INT8), ensuring a fair comparison at the same effective storage cost (8.25 bits/element). INT8 uses per-block symmetric scaling (amax/127). All PTQ comparisons use simple quantize-dequantize round-trips without calibration data or learned scaling — the goal is to compare *format-level* accuracy at equal complexity, not to compete with calibration-based methods (GPTQ, AWQ, QuIP#) that achieve near-lossless results even at 4 bits by optimizing the quantization grid per-layer. Code and raw results are available at https://github.com/Noah-Everett/QuakeFloat8.

SQNR on Pretrained Weights

We apply each format's block-scaled quantization round-trip to all 48 weight tensors (2D+, excluding embeddings) of pretrained GPT-2 Small (124M parameters) and measure SQNR per tensor.

The measured +6.5 dB advantage over MXFP8 is consistent with the structural prediction of ∼6.0 dB from the levels-per-octave ratio ($16/8 = 2$, so $10\log_{10}(4) = 6.0$ dB; Section 6.2), with the additional ∼0.5 dB attributable to FP8's within-binade non-uniformity (Table 3). The advantage shows remarkably low variance across all 48 layers (range: 6.2–6.5 dB). INT8 achieves the highest absolute SQNR (44.6 dB) because it minimizes MSE rather than NMSE — a distinction we revisit below.

Cross-Model Equalization Experiment

The equalization property (Theorem 3.5, Step 1) predicts that QF8's SQNR should be independent of the weight distribution. To test this, we measure per-layer SQNR across 15 pretrained models spanning 5 architectures, 7 model families, and parameter counts from 60M to 1.3B:

Table 9: Log-domain break-even across bit widths.

Bit-width	Mult saved (gates)	LUT overhead (gates)	Net	vs INT-N
4	38	120	−82	1.5× worse
6	130	180	−50	1.2× worse
8	**364**	**300**	+64	0.92× **(slight win)**
12	1,145	500	+645	0.63×
16	2,925	800	+2,125	0.40×

Table 10: Per-layer SQNR on GPT-2 Small pretrained weights (48 tensors).

Format	Mean SQNR	Min	Max
QF8	38.0 dB	37.8 dB	38.1 dB
MXFP8 E4M3	31.5 dB	31.4 dB	31.6 dB
INT8	44.6 dB	41.0 dB	45.4 dB
QF8 − MXFP8	+6.5 dB	+6.2 dB	+6.5 dB

Models tested: GPT-2 (Small, Medium, Large), OPT (125M, 350M, 1.3B), Pythia (160M, 410M, 1B), BERT (Base, Large), RoBERTa-Base, DistilBERT, T5 (Small, Base). All 15 models are transformer variants (decoder-only, encoder-only, or encoder-decoder); CNNs, RNNs, and diffusion models are not represented. The equalization property is mathematically distribution-independent, so it should hold for any architecture, but the empirical confirmation here is limited to transformer weight distributions.

QF8's SQNR standard deviation across *all* 1,481 tensors is 0.10 dB — 5× lower than INT8's 0.54 dB. Figure 1 makes this visible: QF8 collapses to a single spike, while INT8 is spread across a ∼9 dB range.

The kurtosis correlation (Figure 2, Table 11) quantifies dependence on weight distribution shape: INT8's SQNR correlates with kurtosis ($r = -0.49$), meaning its quality degrades for heavy-tailed distributions. QF8's correlation is $r = -0.08$ — effectively zero. This confirms the equalization property: QF8's quantization error is distribution-independent.

Practical consequence: QF8 weight quantization requires no per-layer calibration. The SQNR is guaranteed to be ∼38 dB regardless of the model architecture, training procedure, or weight distribution shape.

Post-Training Quantization on WikiText-2

We evaluate PTQ perplexity on the WikiText-2 test set across three GPT-2 model sizes. For **weight-only** PTQ, all linear layer weights undergo a quantization round-trip before inference. For **W8A8**, activations (inputs to each `nn.Linear`) are additionally quantized via dynamic block scaling at inference time using forward pre-hooks.

Weight-only PTQ (GPT-2). QF8 consistently matches or outperforms MXFP8 across all three GPT-2 sizes, with degradation ≤0.2% everywhere. At GPT-2 Medium (355M), the gap is most pronounced: QF8 degrades by only −0.01% vs. MXFP8's +0.56%.

Table 11: Cross-model SQNR stability (1,481 weight tensors from 15 models).

Format	Mean SQNR	Std Dev	Range	Kurtosis corr. (r)
QF8	**38.1 dB**	**0.10 dB**	3.5 dB	−0.08
MXFP8 E4M3	31.5 dB	0.13 dB	4.8 dB	+0.06
INT8	45.1 dB	0.54 dB	8.9 dB	−0.49

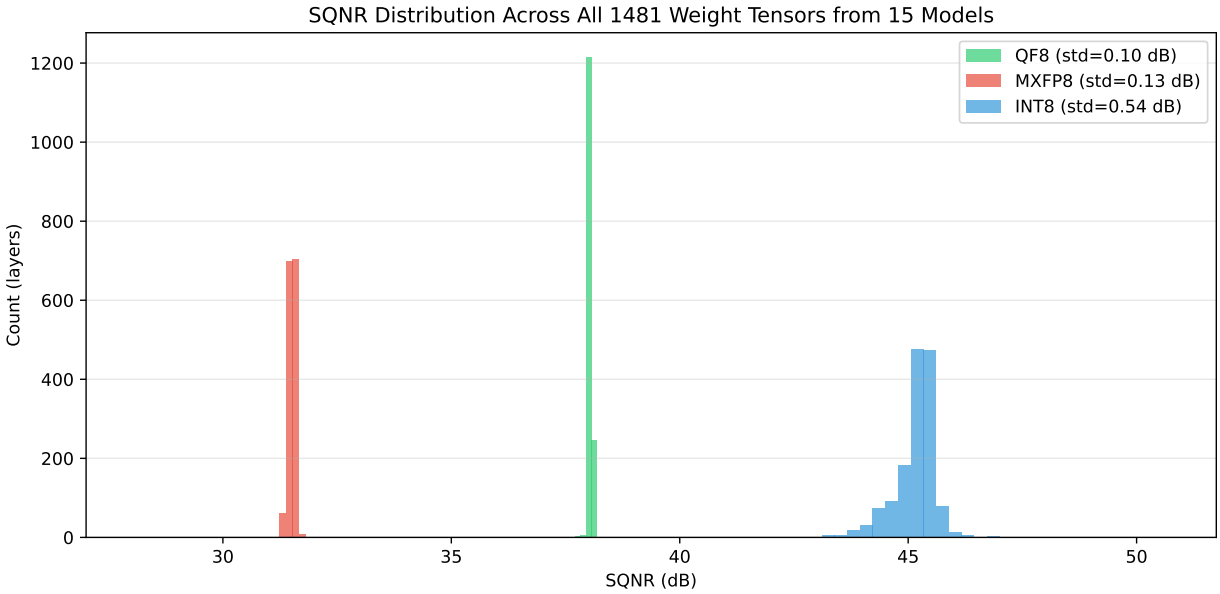

Figure 1: SQNR distribution across all 1,481 weight tensors from 15 models. QF8 (green) is a single spike at 38.1 dB ($\sigma = 0.10$ dB). MXFP8 (red) clusters at 31.5 dB with 3× more variance. INT8 (blue) spreads over a ∼9 dB range, reflecting distribution-dependent error.

Scaling to 7B Parameters

To validate that QF8's advantage holds at the scale where quantization is most relevant, we evaluate weight-only PTQ on three additional models up to 7B parameters. All models are fully open-weight and use the same block-scaled quantization protocol.

At 7B scale, QF8's weight-only degradation is ≤0.03% — effectively lossless. MXFP8's degradation is 10–30× larger (+0.19% to +0.28%). INT8 performs well on absolute perplexity but, as shown in Section 8.2, its SQNR varies significantly across layers and models — it offers no distribution-independent guarantee.

These results use single evaluation runs per configuration. The perplexity differences at this scale (0.002–0.02 perplexity points for QF8) are small enough that they should be interpreted as "effectively lossless" rather than as precise measurements.

W8A8 PTQ: a known limitation. Quantizing activations exposes a structural weakness of log-uniform spacing. At GPT-2 Small (124M), QF8's W8A8 penalty is +1.73% — significantly worse than MXFP8's +0.22% (8× larger). At GPT-2 Medium (355M), QF8 is still 2.2× worse than MXFP8 (+1.43% vs. +0.65%). The cause is fundamental: log-uniform spacing allocates precision uniformly across magnitudes, but post-GELU and post-LayerNorm activations concentrate near zero, where the log grid's resolution is coarsest. At GPT-2 Large (774M), QF8's W8A8 penalty

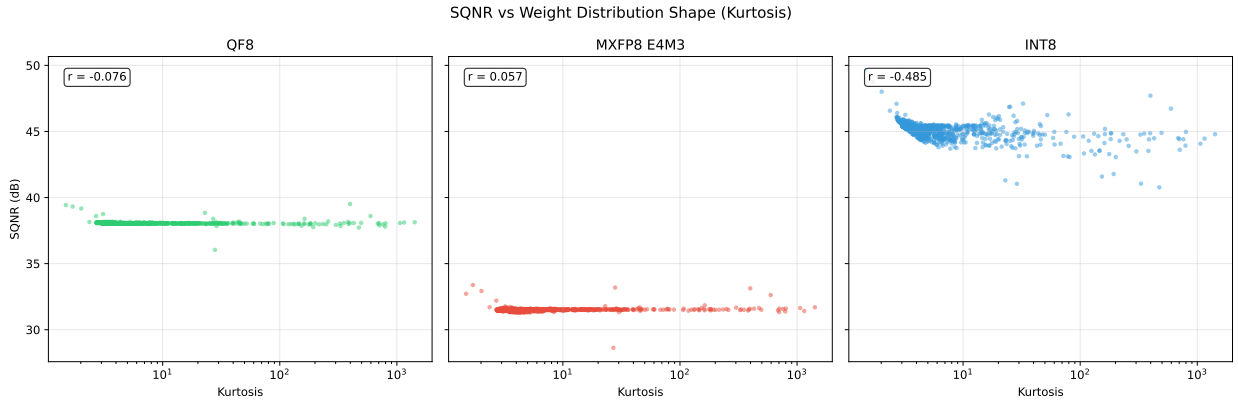

Figure 2: SQNR vs. weight distribution kurtosis (log scale). QF8's correlation is $r = -0.08$ (flat line — distribution-independent). INT8's correlation is $r = -0.49$ (quality degrades for heavy-tailed distributions).

Table 12: Weight-only PTQ perplexity on WikiText-2 (sliding window, stride 512).

Model	Format	Perplexity	Δ vs FP32
GPT-2 Small (124M)	FP32 (baseline)	24.997	—
	QF8	**24.962**	**−0.14%**
	MXFP8 E4M3	25.033	+0.14%
	INT8	25.025	+0.11%
GPT-2 Medium (355M)	FP32 (baseline)	18.243	—
	QF8	**18.241**	**−0.01%**
	MXFP8 E4M3	18.345	+0.56%
	INT8	18.246	+0.02%
GPT-2 Large (774M)	FP32 (baseline)	15.436	—
	QF8	15.453	+0.11%
	MXFP8 E4M3	15.467	+0.20%
	INT8	15.433	−0.02%

is +0.11% (perplexity 15.453 vs. weight-only 15.453; the values differ at the fourth decimal place, 15.4534 vs. 15.4525, and appear identical only due to rounding). This is comparable to MXFP8's +0.23%, but with only three model sizes, single-seed evaluation, and no error bars, this apparent scaling trend is not statistically robust.

Near-zero representation cliff. Within a single block, QF8's smallest nonzero magnitude is $2^{(1-64)/16} \approx 0.065$ (code $c = 1$). There is no gradual underflow: values jump discontinuously from exact zero ($c = 0$) to 0.065. IEEE formats avoid this via denormals, but QF8's fixed-point log encoding has no subnormal region. In practice, E8M0 block scaling mitigates this: the block scale s shifts the smallest nonzero value to $0.065 \cdot s$, which can be as small as $\sim 3.8 \times 10^{-40}$ ($s = 2^{-127}$). The cliff matters only *within* a block where large and near-zero values coexist — precisely the activation pattern produced by ReLU, where the block scale is set by the largest element and the near-zero activations fall into the gap. This is the same root cause as the W8A8 penalty above.

Implication: QF8's theoretical "no calibration" advantage applies to *weight* quantization. For

Table 13: W8A8 PTQ perplexity on WikiText-2 (weights + activations quantized).

Model	Format	Perplexity	Δ vs FP32
GPT-2 Small (124M)	FP32 (baseline)	24.997	—
	W8A8 QF8	25.429	+1.73%
	W8A8 MXFP8	25.053	+0.22%
	W8A8 INT8	26.310	+5.25%
GPT-2 Medium (355M)	FP32 (baseline)	18.243	—
	W8A8 QF8	18.504	+1.43%
	W8A8 MXFP8	18.361	+0.65%
	W8A8 INT8	18.594	+1.93%
GPT-2 Large (774M)	FP32 (baseline)	15.436	—
	W8A8 QF8	15.453	+0.11%
	W8A8 MXFP8	15.472	+0.23%
	W8A8 INT8	15.433	−0.02%

W8A8 deployment, activation-aware techniques (e.g., SmoothQuant-style channel balancing) would likely be needed, partially offsetting the calibration-free benefit.

Quantization-Aware Training

We train GPT-2 Small (162M parameters) from scratch on OpenWebText (∼9B tokens) with STE-based quantization-aware training (W8A8: both weights and activations quantized via block-scaled round-trip at each forward pass, with straight-through gradient estimation). Training uses the GPT-2 BPE tokenizer (50,257 vocab), sequence length 1,024, effective batch size 262K tokens/step (micro-batch 32, gradient accumulation 8), learning rate 3×10^{-4} with cosine decay, on NVIDIA H200 GPUs. FP32 was trained with 3 seeds; QF8 with 1 seed (matched to FP32 seed 42). Training was terminated by cluster walltime before convergence (val loss still declining).

At 40,000 matched steps (the last checkpoint before QF8 was terminated), QF8's validation loss is 3.1310 vs. FP32's 3.1305 (same seed) — a delta of +0.0005 (+0.016%). The three FP32 seeds show cross-seed standard deviation of 0.009, confirming that the +0.016% QF8 gap is well within seed-to-seed noise. At this point in training the model has processed ∼11B tokens and reached validation loss ∼3.13 (perplexity ∼23), substantially beyond the early-training regime.

Limitations. QF8 has only 1 seed (no error bars); FP8 was not included in this run. Training was terminated by cluster walltime at 27% of the target (FP32 reached 54K/200K steps, QF8 reached 43K/200K). The model is not fully converged (val loss was still declining at termination), so this result demonstrates training compatibility through ∼11B tokens, not final converged quality. Multi-seed QAT to full convergence remains future work (Section 10).

Synthetic Benchmarks

To confirm the distribution-independence predicted by the equalization property, we measure SQNR on random matrices and synthetic distributions:

The +6.6 dB advantage is consistent across all tested distributions and matrix sizes, confirming that it is a structural property of the format (the equalization property) rather than a distribution-specific artifact.

Table 14: Weight-only PTQ perplexity at 7B scale (WikiText-2, stride 512).

Model	Format	Perplexity	Δ vs FP16
Phi-2 (2.7B)	FP16 (baseline)	8.646	—
	QF8	**8.646**	**+0.01%**
	MXFP8 E4M3	8.670	+0.28%
	INT8	8.649	+0.04%
Mistral-7B (7B)	FP16 (baseline)	4.798	—
	QF8	**4.798**	**+0.01%**
	MXFP8 E4M3	4.809	+0.23%
	INT8	4.798	+0.01%
Pythia-6.9B (6.9B)	FP16 (baseline)	8.351	—
	QF8	**8.354**	**+0.03%**
	MXFP8 E4M3	8.367	+0.19%
	INT8	8.357	+0.06%

Table 15: GPT-2 Small QAT on OpenWebText (162M parameters, H200 GPUs). FP32: 3 seeds × 54K steps (∼14B tokens). QF8: 1 seed × 43K steps (∼11B tokens). Val loss measured on held-out 5K-document split. Training was terminated by walltime; val loss was still declining.

Format	Val Loss at 40K steps	Δ vs FP32	Steps reached
FP32 (3 seeds)	3.127 ± 0.009	—	54,000
QF8 (seed 42)	3.131	+0.016%	43,000
FP32 seed 42	3.131	—	54,000
QF8 seed 42	3.131	+0.016%	43,000

Literature Positioning

Relationship to Johnson (2018)

The closest prior work is Johnson's "Rethinking Floating Point for Deep Learning" (arXiv:1811.01721), which proposed ELMA: an 8-bit format using log-domain multiply with Kulisch accumulation. QF8's core mechanism — log-domain multiply via integer addition with linear accumulation — *is* Johnson's ELMA.

QF8 should be framed as building on Johnson's architecture with three improvements: (a) fixed-width encoding for hardware simplicity, (b) block scaling for precision/range separation, (c) formal minimax optimality analysis.

Relationship to MX Formats (OCP Microscaling)

QF8 borrows the E8M0 block scaling philosophy directly from the OCP MX specification (Rouhani et al., 2023). The difference is the per-element encoding: MXFP8 uses IEEE-like $4e + 3m$ (8 levels/octave); QF8 uses a log code (16 levels/octave). **QF8 is "MX block scaling + logarithmic per-element encoding."**

Table 16: Matrix multiplication SQNR across sizes (vs. float64 ground truth).

Size	bfloat16	float32	FP8 E4M3 (block)	**QF8**
$16 \times 32 \times 16$	52.3 dB	139.0 dB	28.6 dB	**35.3 dB**
$64 \times 128 \times 64$	52.5 dB	133.6 dB	28.4 dB	**35.1 dB**
$128 \times 256 \times 128$	52.6 dB	130.8 dB	28.5 dB	**35.1 dB**

Table 17: SQNR across value distributions.

Distribution	FP8 E4M3 (block)	QF8	QF8 advantage
$\mathcal{N}(0, 0.02^2)$ — weights	31.4 dB	38.2 dB	+6.7 dB
$\mathcal{N}(0, 1)$ — post-LN acts	31.6 dB	37.9 dB	+6.3 dB
$\text{LogN}(0, 1)$ — gradients	31.5 dB	38.3 dB	+6.8 dB
$\text{Laplace}(0, 0.02)$ — sparse	31.5 dB	38.0 dB	+6.5 dB
90% sparse + $\mathcal{N}(0, 1)$	31.7 dB	38.3 dB	+6.6 dB

Relationship to NF4 (QLoRA)

Both NF4 (Dettmers et al., 2023) and QF8 exploit distribution-aware encoding. NF4 places levels at Gaussian quantiles (optimal for a specific distribution); QF8's log-uniform spacing is minimax-optimal across all distributions. NF4 is a storage format (dequantize before compute); QF8 is a computational format (arithmetic in log domain).

Logarithmic Number Systems

QF8 is a descendant of the Logarithmic Number System (LNS), known since the 1970s (Kingsbury & Rayner, 1971; Swartzlander & Alexopoulos, 1975). The fundamental idea of log-domain multiplication is not new. QF8's novelty is the specific combination of format design, block scaling, and formal optimality analysis for ML workloads.

What Is Genuinely Novel

1. **The equalization property (Theorem 3.5, Step 1).** The log-uniform quantizer achieves $\text{MSRE} = \varepsilon^2/12$ for *any* source density f. This density-independence follows from the exact x^2 cancellation in the NMSE ratio — a structural property absent in all prior companding results. Smith [15] observed amplitude-independent SNR for log-PCM, but his result is about signal level, not source distribution, and his distortion formula remains distribution-dependent.

2. **Fixed-rate minimax optimality.** As a consequence of (1), log-uniform uniquely minimizes worst-case NMSE over all source densities. Sun & Goyal [16] prove log companding is optimal for variable-rate MSRE quantization but show it is *not* generally optimal for fixed-rate. Theorem 3.5 provides the complementary fixed-rate minimax result — which holds precisely *because* of (1). No prior 8-bit ML format has any proof of optimality.

3. **The specific format:** `u3.4` log code + E8M0 block scaling + $O(1)$ bit-trick decode.

4. **Quantified advantage:** +6.6 dB / 4.6× NMSE improvement over FP8 E4M3 at identical storage cost (8.25 bits/element). The advantage is mathematical: it follows from 16 vs. 8 levels per octave.

Table 18: QF8 vs. Johnson's ELMA.

Feature	Johnson ELMA (8-bit)	QF8
Log fractional bits	∼6 (posit tapered)	4 (fixed `u3.4`)
Encoding	Posit regime bits (variable)	Fixed-width `u3.4`
Block scaling	None (self-scaled)	E8M0 per 32 elements
LUT size	64 entries (2^6)	16 entries (2^4) — 4× smaller
Optimality proof	None	Minimax NMSE (Theorem 3.5)

5. **Empirical confirmation of equalization:** Across 1,481 weight tensors from 15 models and 5 architectures, QF8's SQNR is 38.1 ± 0.10 dB with near-zero dependence on weight distribution shape ($r = -0.08$ vs. kurtosis). Weight-only PTQ at 7B scale shows ≤0.03% perplexity degradation.

6. **Hardware cost analysis:** Estimated ∼2× cheaper full MAC unit than FP8 (650 vs. 1350 gates), with the multiply path itself 3.5× cheaper (66 vs. 231 gates). These are NAND2-equivalent estimates, not synthesis results.

The One-Sentence Pitch

> *QF8 is what you get when you take Johnson's log-domain multiply architecture, replace the posit tapered encoding with a simpler fixed-width log code, add MX-style block scaling, and prove that the resulting format is minimax-optimal — yielding 2× the precision per octave of FP8 at comparable hardware cost.*

Future Work

Remaining Gaps

Weight-only PTQ results from 124M to 7B parameters (Section 8.3.1) and the cross-model equalization experiment (Section 8.2) validate QF8 for inference-time weight quantization across diverse architectures and scales. Several gaps remain:

1. **W8A8 activation handling.** QF8's log-uniform grid is structurally suboptimal for near-zero-concentrated activations (Section 8.3). Activation-aware calibration (e.g., SmoothQuant-style channel balancing) is the most promising mitigation, but would reintroduce per-layer calibration — partially offsetting the format's calibration-free appeal.

2. **QAT at scale.** The GPT-2 Small experiment (162M parameters, 20,000 steps on OpenWebText) shows near-zero degradation (+0.02%) but is single-seed. Multi-seed QAT at 1B+ scale is needed to establish training robustness.

3. **Scaling beyond 7B.** Weight-only PTQ at 13B–70B would strengthen the scaling story. The equalization property predicts identical SQNR at any scale; confirming this for perplexity at 70B would be definitive.

4. **RTL synthesis.** Hardware gate counts are NAND2-equivalent estimates. Actual synthesis at 7nm or 28nm would validate (or correct) the claimed area and power advantages.

Gradient Quantization

QF8's `u3.4` encoding gives 8 octaves per element, which may be insufficient for gradients (which need wider dynamic range than weights/activations). A `u4.3` variant (4 integer + 3 fractional bits = wider range, lower precision) for backward passes is a natural extension but has not been validated. FP8 training uses E5M2 for gradients for the same reason.

BMD Decoder Conjecture

Conjecture 10.1 (BMD Decoder Characterization)**.** *The class of monotone BMD decoders from B-bit integers to positive reals is exactly the class of functions expressible as $\hat{D}(n) = \text{float_reinterpret}(a \cdot n + b)$ for integer constants a, b, or compositions of a bounded number of such reinterpretations with integer arithmetic.*

This is speculative: we have no proof, no counterexample, and no evidence beyond the observation that known BMD decoders (LUT, Quake trick, quadratic approximation) all fit this pattern. We include it as an open question that may interest the hardware/arithmetic community.

Appendix: Rate-Distortion Context

Gaussian Source Rate-Distortion

For $X \sim \mathcal{N}(0, \sigma^2)$:

$$R(D) = \frac{1}{2} \log_2 \left(\frac{\sigma^2}{D} \right), \quad D \leq \sigma^2$$

This yields the **6 dB/bit rule**: each additional bit reduces MSE by 4× (6.02 dB).

Scalar Quantization Gap (Zador)

The maximum improvement from scalar to infinite-dimensional vector quantization:

$$\frac{G_1}{G_\infty} = \frac{\pi e}{6} \approx 1.42 \quad (1.53 \text{ dB})$$

This modest gap means a well-designed scalar quantizer like QF8 comes within 1.53 dB of the VQ optimum.

Bit Allocation

For tensor types $t \in \{w, a, g\}$ with sensitivity weights c_t and distribution variances σ_t^2, the optimal allocation minimizes: $\sum_t c_t \sigma_t^2 \cdot 2^{-2B_t}$ subject to $\sum_t B_t = B_{\text{total}}$.

Solution (reverse water-filling):

$$B_t^* = \frac{B_{\text{total}}}{|\mathcal{T}|} + \frac{1}{2} \log_2 \left(\frac{c_t \sigma_t^2}{G} \right), \qquad G = \left(\prod_t c_t \sigma_t^2 \right)^{1/|\mathcal{T}|}$$

With corrected parameters ($c_w = 1, c_a = 1, c_g = 10^{-6}$), the near-zero gradient allocation ($B_g = 0.18$) reflects that $c_g = \eta^2 \approx 10^{-6}$ dramatically reduces gradient sensitivity, motivating stochastic rounding or shared-exponent compression for gradients.

References

[1] Johnson, J. (2018). "Rethinking Floating Point for Deep Learning." arXiv:1811.01721.

[2] Micikevicius, P. et al. (2022). "FP8 Formats for Deep Learning." arXiv:2209.05433.

[3] Rouhani, B. et al. (2023). "Microscaling Data Formats for Deep Learning." arXiv:2310.10537.

[4] Dettmers, T. et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." arXiv:2305.14314.

[5] Gustafson, J. & Yonemoto, I. (2017). "Beating Floating Point at its Own Game: Posit Arithmetic." Supercomputing Frontiers and Innovations 4(2):71–86.

[6] Horowitz, M. (2014). "Computing's Energy Problem (and what we can do about it)." ISSCC.

[7] Jouppi, N. et al. (2017). "In-Datacenter Performance Analysis of a Tensor Processing Unit." ISCA.

[8] Kingsbury, N. & Rayner, P. (1971). "Digital filtering using logarithmic arithmetic." Electronics Letters.

[9] Swartzlander, E. & Alexopoulos, A. (1975). "The Sign/Logarithm Number System." IEEE Trans. Computers.

[10] Miyashita, D. et al. (2016). "Convolutional Neural Networks using Logarithmic Data Representation." arXiv:1603.01025.

[11] Sze, V. et al. (2017). "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." Proc. IEEE.

[12] Kuzmin, A., Van Baalen, M., Ren, Y., Nagel, M., Peters, J., Blankevoort, T. (2022). "FP8 Quantization: The Power of the Exponent." arXiv:2208.09225.

[13] Bennett, W.R. (1948). "Spectra of Quantized Signals." *Bell System Technical Journal* 27(3):446–472.

[14] Panter, P.F. and Dite, W. (1951). "Quantization Distortion in Pulse-Count Modulation with Nonuniform Spacing of Levels." *Proceedings of the IRE* 39(1):44–48.

[15] Smith, B. (1957). "Instantaneous Companding of Quantized Signals." *Bell System Technical Journal* 36(3):653–709.

[16] Sun, J.Z. and Goyal, V.K. (2011). "Scalar Quantization for Relative Error." *Proc. IEEE Data Compression Conference (DCC)*, pp. 293–302.

[17] Gray, R.M. and Neuhoff, D.L. (1998). "Quantization." *IEEE Trans. Information Theory* 44(6):2325–2383.

[18] Gersho, A. and Gray, R.M. (1992). *Vector Quantization and Signal Compression.* Kluwer Academic.